



Computer Vision

Trabajo Práctico Final

Sancho Almenar, Mariano S-5778/9

29/06/2025

Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Tecnicatura Universitaria en Inteligencia Artificial

1. Introducción.....	3
2. Estructura del proyecto y consideraciones previas.....	3
3. Análisis del Dataset.....	4
4. Etapa 1: Buscador de Imágenes por Similitud.....	5
4.1 Creación de la Base Vectorial.....	5
5. Etapa 2: Entrenamiento y Comparación de modelos Clasificación.....	6
5.1. ResNet18 - Base.....	7
5.2. ResNet18 - FineTuning.....	8
5.3 CNN Custom.....	10
5.4 Comparación entre modelos.....	11
5.5 Interfaz de Gradio.....	11
6. Etapa 3: Pipeline de Detección y Clasificación de Escenas Complejas.....	12
6.1 Elección de Modelo.....	12
6.2 Inferencia.....	13
7. Etapa 4: Evaluación, Optimización y Herramientas de Anotación.....	14
7.1 Evaluación.....	14
8. Conclusiones finales.....	15

1. Introducción

El siguiente trabajo abarca el uso de diferentes modelos para la detección y clasificación de distintas razas de perros a partir de un conjunto de imágenes. Con el correr de la resolución de cuatro etapas preestablecidas, se construyen aplicaciones capaces de realizar búsqueda por similitud de imágenes, clasificación supervisada, detección de escenas complejas y generación automática de anotaciones.

Se presenta el uso de modelos pre entrenados conocidos por su gran efectividad, como lo son ResNet para la clasificación de imágenes, YOLO para la detección y clasificación de objetos (en este caso perros) y redes convolucionales personalizadas además de herramientas tales como Gradio para desarrollo de apps simples, FAISS para la creación de base vectorial y Pytorch como framework principal para el manejo de los modelos de aprendizaje.

El objetivo es la construcción y evaluación de estos modelos, permitiendo así una comparación entre cada uno y llegar a una conclusión final sobre cuál fue el más efectivo en términos de resultados y tiempo de ejecución.

2. Estructura del proyecto y consideraciones previas

El repositorio incluye todos requerimientos necesarios para su ejecución, tanto de forma local como en plataformas en línea como Google Colab.

Al ejecutar el notebook por primera vez, se descarga y descomprime automáticamente la carpeta *data*, que contiene el dataset con imágenes de 70 razas de perros. También se incluyen las siguientes carpetas adicionales:

- *complex_images*: utilizada en la Etapa 4 para evaluar el pipeline completo en escenas reales.
- *test_yolo*: empleada únicamente para comparar resultados entre distintos modelos de YOLO.

El repositorio contará además con carpetas destinadas para el guardado de los plots y modelos generados.

Para el desarrollo del trabajo se tuvo en cuenta una metodología funcional, priorizando siempre que fuese posible la reutilización de código y la modularidad en el diseño.

Todas las mediciones expresadas en el informe corresponden a los modelos generados en la última ejecución del notebook al momento de realizar la entrega. Los mismos se encuentran disponibles en la carpeta *models*.

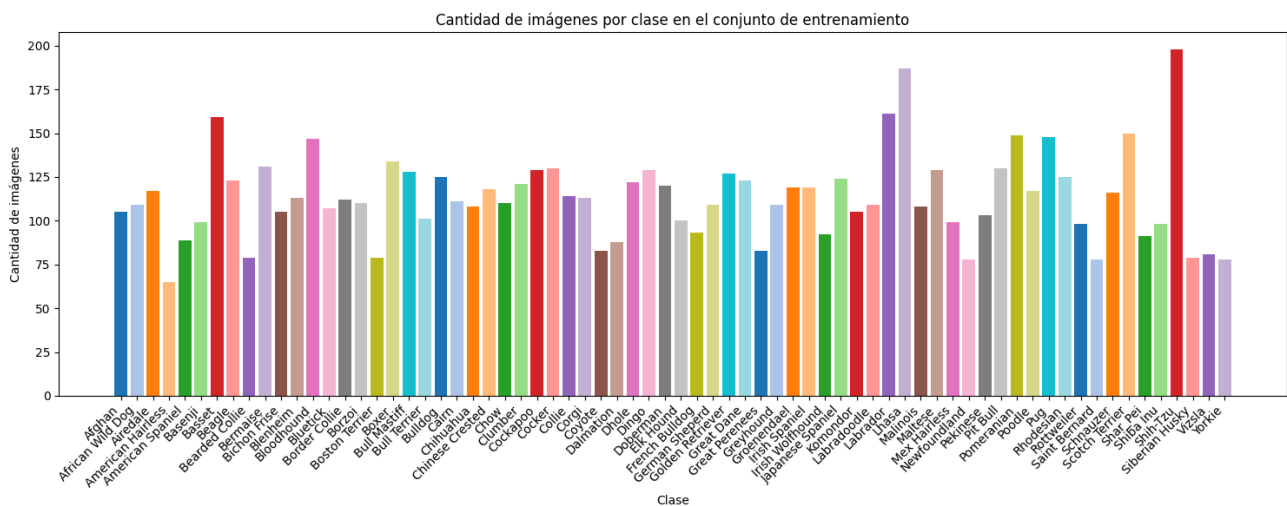
3. Análisis del Dataset

Como ya se ha dicho anteriormente, contamos con un dataset con 70 razas de perros. Una vez descargada y descomprimida la carpeta, procedo a realizar un análisis simple sobre el mismo para entender de mejor manera sobre que estaré trabajando.

El dataset en su totalidad cuenta con **9346** imágenes repartidas en:

- **7946** en el conjunto de train
- **700** en el conjunto de validación
- **700** en el conjunto de test

Realizo un análisis sobre el balanceo entre clases y obtengo el siguiente resultado:



Se observa que la mayoría de las clases se encuentran en el rango de entre 75 y 130 imágenes por clase, aunque se presentan algunos extremos con más de 190 y otras con menos de 70.

Se podría considerar que el dataset presenta un desbalance, es por eso que al momento de generar el trainset, se aplica una transformación extra de data augmentation.

A todas las imágenes se les realiza una transformación de su tamaño, ajustándose por defecto a (224,224).



4. Etapa 1: Buscador de Imágenes por Similitud

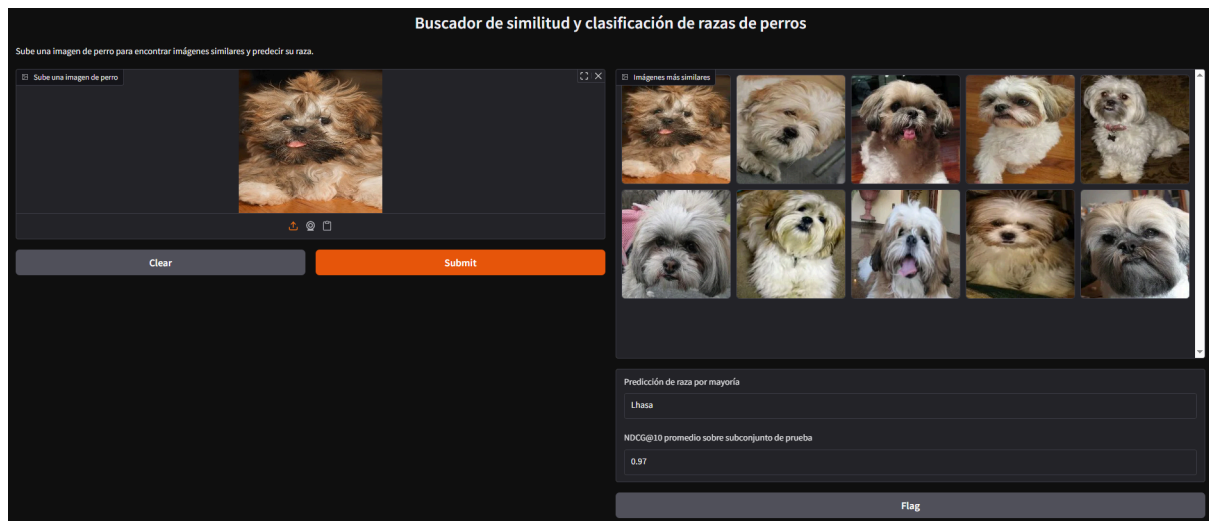
En la primera etapa del proyecto se busca crear una aplicación capaz de encontrar imágenes similares en el dataset a partir de una imagen ingresada por el usuario.

En esta etapa es además donde se comienza a formar la estructura y la utilización de determinados frameworks y librerías. Desde un comienzo, la intención fue utilizar el modelo pre-entrenado ResNet 50 usando el framework de Tensor Flow para su implementación. Sin embargo, esta decisión resultó poco adecuada, ya que a medida que el proyecto avanzaba fue necesario integrar otras librerías cuya compatibilidad con TensorFlow presentaba inconvenientes. Esto afectó negativamente la metodología funcional, ya que obligaba a desarrollar funciones específicas para los modelos implementados en TensorFlow, dificultando la reutilización de código. Por este motivo, se decidió unificar el desarrollo en torno a PyTorch como framework principal, lo que facilitó la integración entre componentes, simplificó el mantenimiento del código y permitió una mayor consistencia en todo el pipeline.

4.1 Creación de la Base Vectorial

En este bloque se construye la base vectorial que permitirá realizar búsquedas por similitud de imágenes. Para ello, se utiliza un modelo ResNet50 pre entrenado en ImageNet, al que se le elimina la última capa de clasificación para obtener únicamente los vectores de características (embeddings) de cada imagen. Estas representaciones se extraen para todas las imágenes del conjunto de entrenamiento, se almacenan junto con sus rutas y etiquetas, y se indexan en una estructura FAISS que permite búsquedas rápidas basadas en distancia euclidiana. Además, se guarda una copia desnormalizada de cada imagen en un directorio temporal para poder recuperarlas luego y mostrarlas como resultados.

Una vez obtenida la base vectorial, se desarrolla una interfaz de Gradio, donde el usuario ingresará una imagen y la aplicación se encargará de extraer el vector de características de la misma usando el modelo para generar los embeddings y consultar a la base vectorial para así poder mostrar en pantalla las 10 imágenes con más similitud. Se agrega además, la raza predicha calculada en base a voto mayoritario, es decir, de las 10 imágenes obtenidas, se calcula cuál fue la raza con más apariciones y la contamos como predicción.



Se calcula además el NDCG que mide qué tan relevantes y bien ordenados están los resultados en la búsqueda, considerando que los elementos más relevantes deben aparecer en los primeros lugares. En este caso, tenemos un NDCG de **0,97** lo que nos indica que el sistema funciona correctamente y que la gran mayoría de imágenes recuperadas pertenece a la clase real de la imagen.

5. Etapa 2: Entrenamiento y Comparación de modelos Clasificación

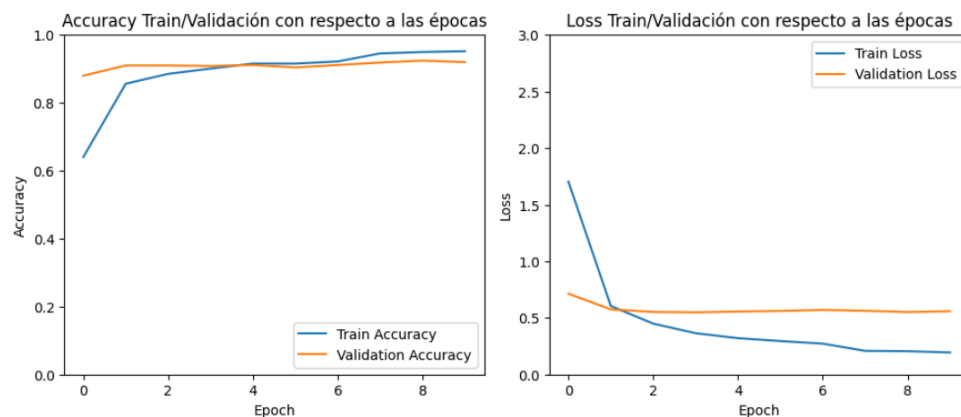
Para esta etapa se emplea el uso de distintas funciones con el fin de reutilizarlas para los modelos utilizados. Entre ellas se encuentran la función de entrenamiento, predicción y generación de métricas.

Los modelos evaluados en este bloque incluyen una versión fine-tuneada de ResNet18 pre entrenada en ImageNet y una red convolucional personalizada diseñada desde cero. Además, se incorpora como punto de comparación el modelo base de ResNet18 sin fine-tuning, con el fin de analizar cuánto mejora el desempeño al adaptar el modelo a las características específicas del dataset.

5.1. ResNet18 - Base

El modelo se entrena por 10 épocas, nos guardamos el mejor modelo cuyo resultados fueron:

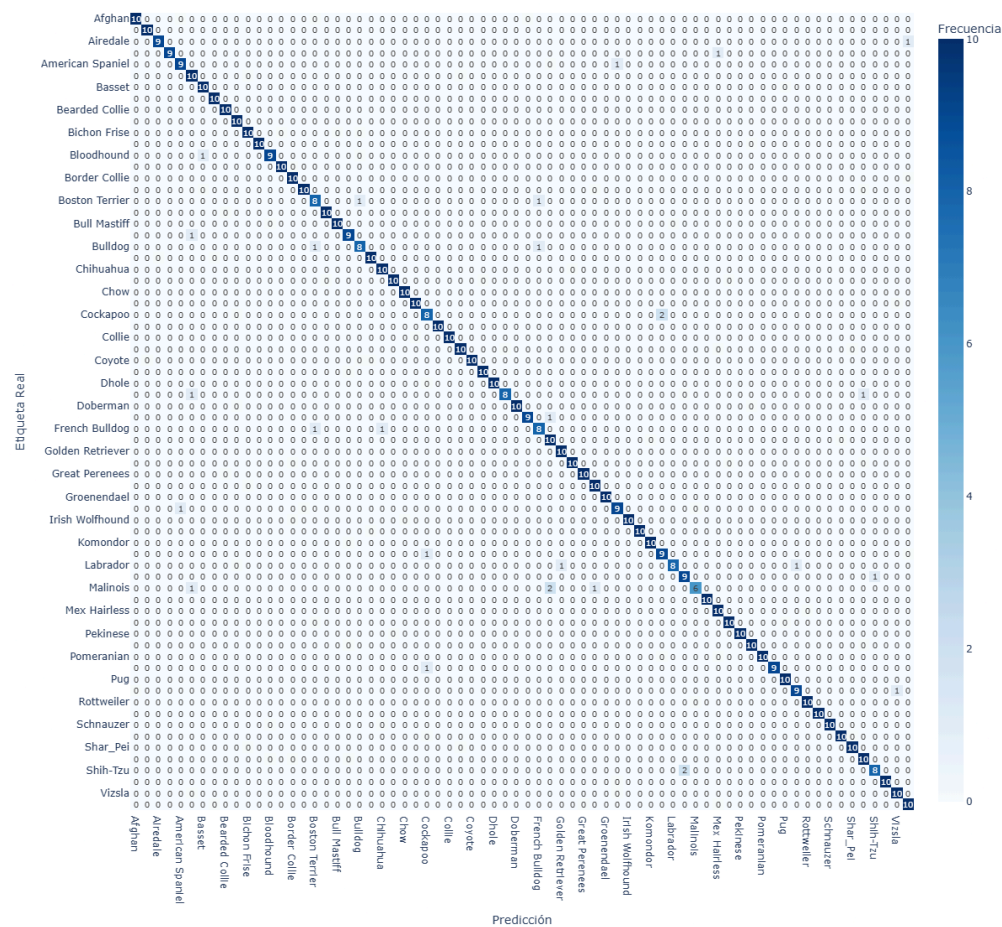
Accuracy	Loss	Val. Accuracy	Val. Loss
0,94	0,20	0,92	0,55



Podemos ver que la curva de Accuracy tanto en Train como en Val. convergen en los ~0.92, lo que indica un buen desempeño. Sin embargo, en la curva de Loss vemos como para Train converge a 0.20 y en Val. no baja de 0.50.

Esto sugiere una posible tendencia al *overfitting* si se continúa entrenando por más épocas. Por este motivo, se considera que no sería beneficioso extender el entrenamiento y, en cambio, podría ser conveniente reducir la cantidad de épocas para optimizar los tiempos de ejecución sin comprometer significativamente la calidad del modelo.

Matriz de Confusión - Modelo Base

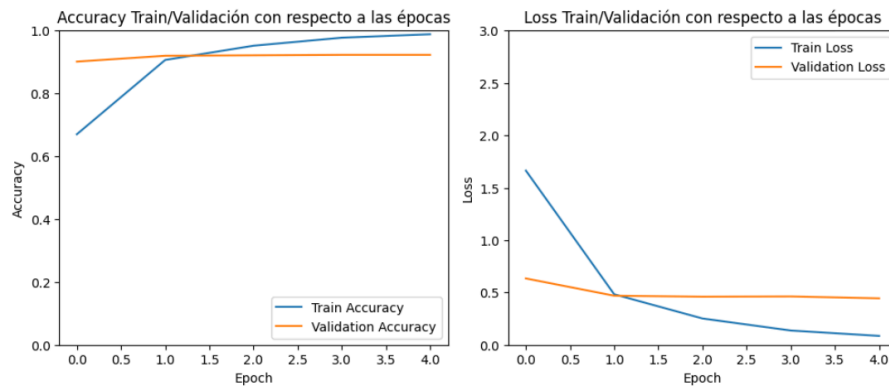


El modelo logra captar y clasificar correctamente todas las clases, con apenas algunas excepciones, posiblemente debido a similitudes entre razas, mala calidad de la imagen o desbalanceo leve de dichas clases

5.2. ResNet18 - FineTuning

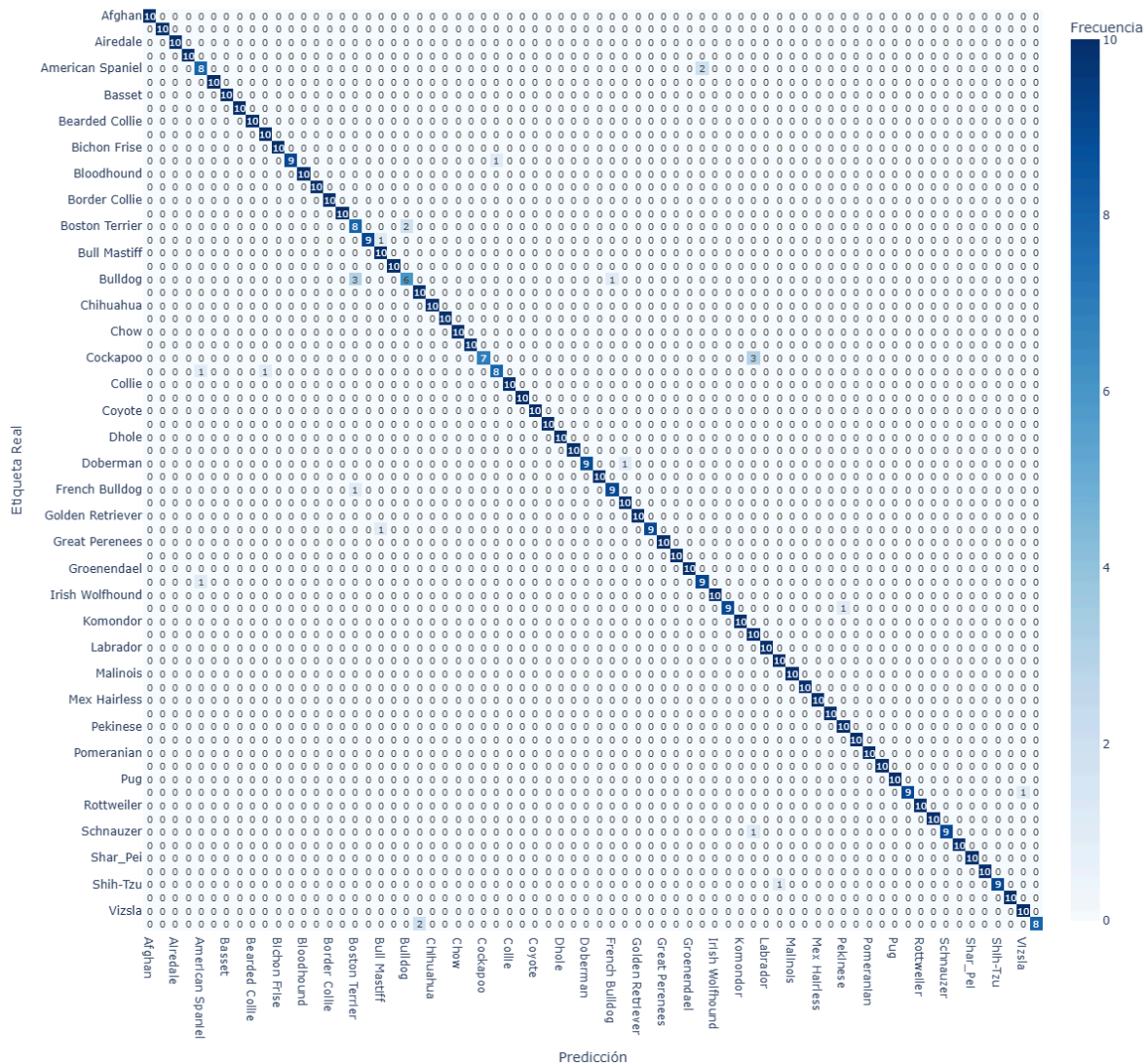
Teniendo en cuenta el buen desempeño del modelo anterior, se procede realizar un fine-tuning y a entrenar sólo por 5 épocas, obteniendo los siguientes resultados:

Accuracy	Loss	Val. Accuracy	Val. Loss
0,98	0,08	0,044	0,92



Se logra mejorar todas las métricas, con el Accuracy de Train llegando a los ~0.99 y Val Accuracy de ~0.93. El loss también se ve reducido aunque no significativamente y sigue la tendencia observada anteriormente al overfitting en caso de entrenar por más épocas.

Matriz de Confusión - ResNet18 Fine Tuned



Es evidente el excelente rendimiento del modelo, mejora en las predicciones del modelo base con apenas algunas predicciones erróneas.

5.3 CNN Custom

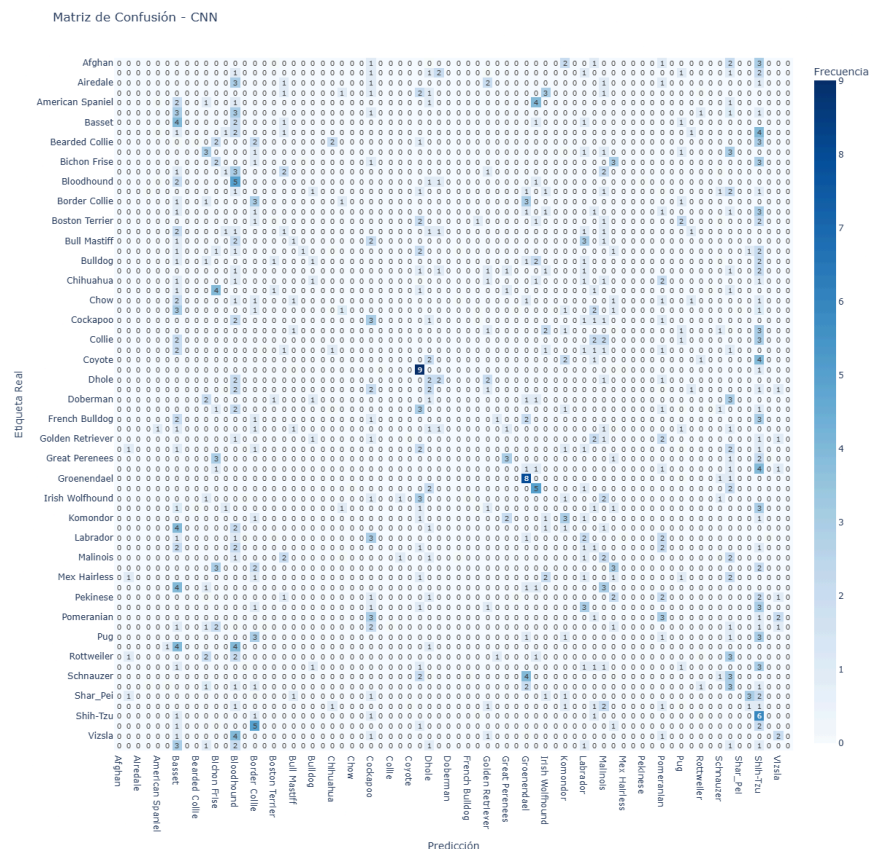
Este fue quizás el principal inconveniente del proyecto, ya que como se explicó anteriormente, abordé un enfoque utilizando Tensor Flow, pero debido a incompatibilidades, se terminó por adaptar lo realizado al framework de PyTorch.

En un principio se construyó una red neuronal usando TF que si bien no lograba las métricas de ResNet, funcionaba mejor que la red que se terminó utilizando en el trabajo.

La red anterior lograba un accuracy 0.6, lo que dado la naturaleza del problema era un resultado esperable. Sin embargo, al desarrollar la red usando Pytorch, logra resultados pobres, asignando la mayoría de valores a aquellas clases con mayor muestra de imágenes.

Entiendo que la red está entrenando de forma errónea y que se debe arreglar para su correcto funcionamiento.

Accuracy	Loss	Val. Accuracy	Val. Loss
0,08	3,76	0,097	3,7



La matriz de confusión generaliza de manera extremadamente pobre con tendencias a predecir por aquellas clases con mayor cantidad de imágenes.

5.4 Comparación entre modelos

Los resultados finales de los modelos fueron los siguientes:

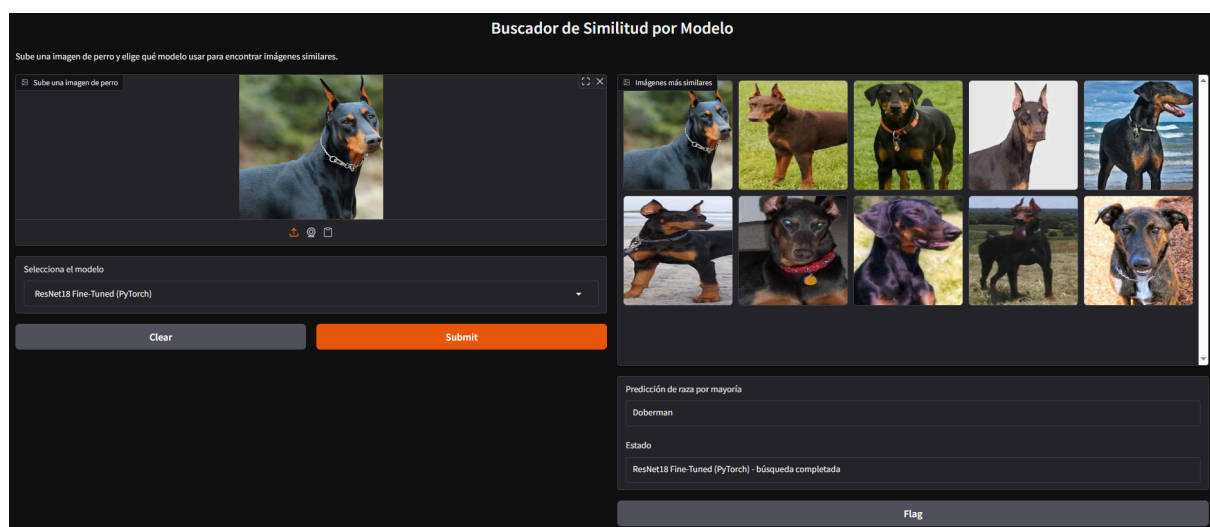
Modelo	Accuracy	Precisión	Recall	F1	Especif.
ResNet18	0,96	0,96	0,96	0,96	1
ResNet18-FT	0,96	0,97	0,96	0,96	1
CNN	0,11	0,07	0,11	0,07	0,99

Los resultados muestran que tanto el modelo ResNet18 base como el fine-tuneado (ResNet18-FT) logran un rendimiento muy alto y comparable, con métricas de precisión, recall y F1-score cercanas al 96–97%, y una especificidad perfecta (1.00), lo cual indica una excelente capacidad de discriminación entre clases.

En el caso de la CNN voy a obviar las métricas obtenidas debido a su entrenamiento erróneo.

5.5 Interfaz de Gradio

Se crea una nueva interfaz, con las mismas funcionalidades que la anterior, ahora realizando la búsqueda por medio del mejor modelo.

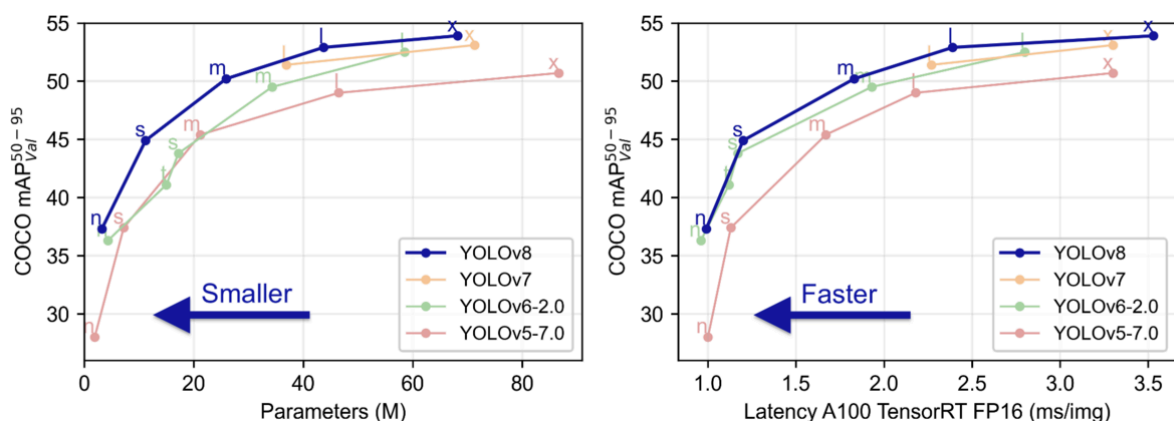


6. Etapa 3: Pipeline de Detección y Clasificación de Escenas Complejas

El objetivo de esta etapa es la de crear un sistema capaz de realizar inferencia sobre imágenes del mundo real, donde puede haber múltiples objetos además de perros. Para ello, se emplea **YOLOv8 con segmentación**, un modelo avanzado que permite detectar e instanciar individualmente cada objeto presente en la imagen, generando tanto las cajas delimitadoras (*bounding boxes*) como las máscaras de segmentación correspondientes.

6.1 Elección de Modelo

Si bien está definido la utilización de YOLOv8, estos modelos se clasifican además de acuerdo a la cantidad de sus parámetros y complejidad.



A medida que se cargue un modelo más grande, los parámetros aumentarán como también su tiempo de inferencia. Es por eso que es necesario evaluar primero qué modelos nos conviene utilizar de acuerdo a las circunstancias.

En mi caso, comparo los dos modelos más básicos, Nano y Small, ya que considero que son lo suficientemente robustos para la tarea a desarrollar.

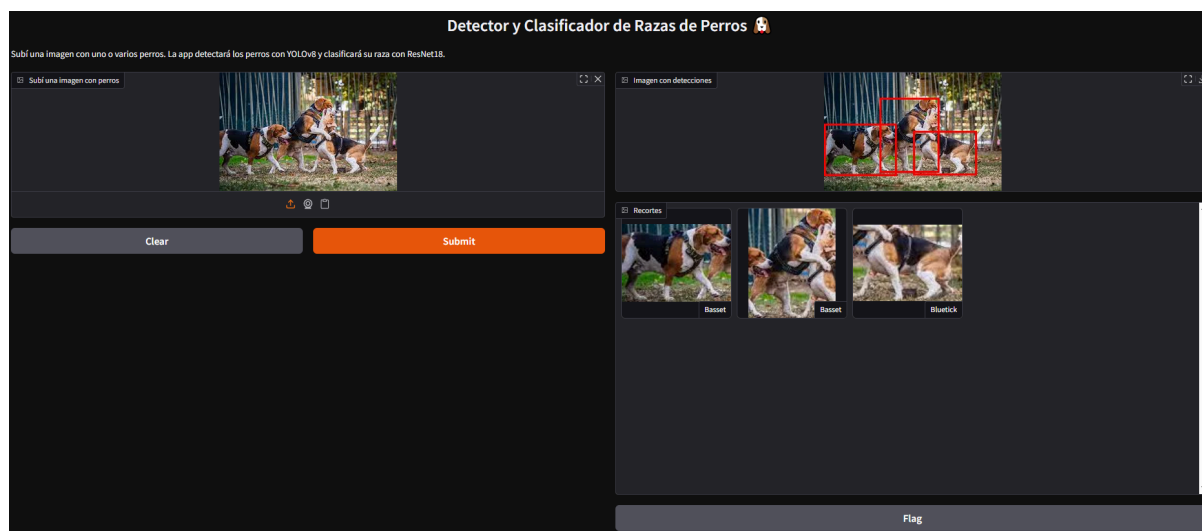


Al hacer una simple comparación sobre dos imágenes complejas, vemos que ambos predicen de manera correcta, sin embargo la versión Small logra un mayor porcentaje de confianza sobre la predicción.

Si analizamos la cantidad de parámetros, el modelo **YOLOv8n (Nano)** posee aproximadamente **3.4 millones**, mientras que **YOLOv8s (Small)** cuenta con **11.8 millones**, lo que representa un incremento de alrededor del **247%** en la cantidad de parámetros. Sin embargo, a pesar de este aumento, en la práctica **no se observa una diferencia significativa en el tiempo de inferencia**, y el modelo Small logra resultados notablemente mejores en términos de precisión y detección. Por este motivo, considero que **YOLOv8 Small es la mejor opción** para este proyecto, ya que ofrece un buen equilibrio entre desempeño y eficiencia.

6.2 Inferencia

Para realizar la inferencia nuevamente se utiliza la interfaz de gradio. En este caso, el usuario ingresará una imagen y el programa detectará con YOLO los perros presentes en las mismas. Creará bounding boxes de cada perro y se los enviará al modelo de ResNet18 fine tuned entrenado anteriormente para realizar la clasificación. En pantalla se mostrará la imagen original, la imagen original con bounding boxes y para finalizar los recortes de cada perro con su raza predicha.



Si bien la evaluación del modelo se realiza en la siguiente etapa, se puede observar que ResNet no está clasificando correctamente imágenes por fuera de los conjuntos de test y validación.

7. Etapa 4: Evaluación, Optimización y Herramientas de Anotación

En la etapa final, se crea un dataset manualmente con 10 imágenes complejas que contengan perros y demás objetos.

El dataset fue creado utilizando el entorno de CVAT, en donde por cada imagen del conjunto se crea manualmente el bounding box del perro y se le asigna su raza.

El modelo, al realizar la inferencia sobre el conjunto de imágenes complejas logra resultados pobres en cuanto a la clasificación. Las métricas obtenidas fueron las siguientes:

mAP	IoU	Precisión	Recall	F1-Score
0,072	0,88	0,35	0,30	0,32

Los resultados obtenidos muestran un **IoU promedio alto (0.88)**, lo que indica que el modelo logra **delimitar correctamente las bounding boxes de los perros** en las imágenes. Sin embargo, métricas como la **precisión (0.35)**, el **recall (0.30)** y el **F1-score (0.32)** son considerablemente más bajas, lo que sugiere que el sistema tiene dificultades para **clasificar correctamente la raza de los perros detectados**. Esto podría deberse a errores del clasificador posterior al recorte, a confusión entre razas similares, o a limitaciones del dataset de evaluación.

En conjunto, el sistema **detecta bien la ubicación de los objetos**, pero presenta margen de mejora en el etiquetado **correcto de las razas**.

8. Conclusiones finales

Considero que el trabajo fue una buena introducción a la visión por computadora. Abarca todo el proceso desde la preparación y anotación de los datos, hasta la implementación de modelos de detección y clasificación. A lo largo del proyecto se utilizaron librerías y frameworks especializados, lo cual permitió el aprendizaje sobre el uso de algunas de las herramientas fundamentales en el área.

Además, los productos generados, como el sistema de anotación automática y la aplicación de búsqueda por similitud, son ejemplos concretos de aplicaciones que podrían ser utilizadas en distintos rubros. Se podría pensar, por ejemplo, en una aplicación forense donde, a partir de un retrato o una imagen capturada, se utilice un sistema de búsqueda por similitud para identificar personas en una base de datos. Este sistema permitiría comparar el retrato o la imagen con fotografías previas, acelerando la identificación de sospechosos o personas de interés.

En tanto a los modelos generados, ResNet18 con fine-tuning fue el modelo que mejor logró detectar las razas mientras se probaba en el conjunto de prueba. Sin embargo, su rendimiento disminuyó a la hora de clasificar con imágenes cotidianas.

Me gustaría resaltar que es un campo en constante evolución, por lo que es fundamental estar atentos a los avances y nuevos modelos que puedan ofrecer mejores en la precisión, rapidez y robustez.